Security Documentation

Dynamics Queue Admin is a dispatch-queue management console (single-page application), version 1.0.0. This is the security submission against the INSA Secure Website Management Standard, 2014 EC.

Document	**SEC-1.0.0**, Security Documentation
Asset name	Dynamics Queue Admin, a queue organization and dispatch management console
Asset type	Web Application (client-side single-page application)
Asset version	1.0.0
Asset code	`ASSET-DYN-QA-001`
Repository	`queadmin-frontend`
Production host	`queue.dynamicsroute.tech`
Backend API	Separate system (`transportBackEndNative`, Node.js/Express + MySQL). Development API `https://dev.dynamicsroute.tech`; production build defaults to the same-origin path `/api` unless `VITE_API_BASE_URL` is supplied at build time.
Governing standard	INSA Secure Website Management Standard, 2014 EC (`INSAREQUIREMENT.md`), Annexes A and B
Submitting entity	Dynamics Transport
Contact person	Marew Masresha, General Manager — `mmarew@gmail.com`, `0983222221`
Date of preparation	15 October 2026
Date of submission	15 October 2026

Honesty note on scope and status

This document covers **this repository and this asset only**: the Dynamics Queue Admin console SPA. The backend (`transportBackEndNative`) and the hosting platform are separate systems, and they are documented elsewhere. Where a control depends on either of them, this document says so rather than claiming the control as ours. Application-layer controls that are present in the submitted source are marked PASS. Server and hosting controls that are **absent entirely** (this repository holds no CSP, no security response headers and no nginx/server configuration), together with organizational items that carry no evidence, are marked OPEN. Genuinely inapplicable tests are marked N/A. Ownership and contact fields are completed by the submitting entity.

Contents

1	Introduction
2	Asset and contact information
3	System description
4	Security requirements and objectives
5	Authentication and session management
6	Authorization and access control
7	Transport security
8	Application security controls
9	Input validation and data handling
10	Third-party and integration security
11	Dependency and supply-chain management
12	Deployment and hosting security
13	Logging, monitoring and incident response
14	Testing and assurance (Annex B checklist)
15	Annex A — Common vulnerabilities and threats
16	Compliance summary and deviations register
17	Security recommendations and roadmap
—	Document control, approvals and signatures

Related documents

Document	Identifier / location
Software Design Description	`SDD-1.0.0`, `docs/insa/SDD.html`
Technical Design / Low-Level Design	`TDD-1.0.0`, `docs/insa/TDD_LLD.html`
End User Guide	`docs/insa/UserGuide.html`
Standard and checklist	`INSAREQUIREMENT.md`

Document	Identifier / location
In-application security self-assessment	`src/pages/security/SecurityPage.tsx` with `src/data/insaStandard.ts` (5 focus areas, 35 requirements, 102 tests, 8 Annex A vulnerabilities)

How to read this document

Section 14 holds the full 102-item INSA Annex B checklist, and every item carries a status. Section 16 maps the five focus areas (2.1–2.5) to what the organization can show today, and Section 17 sets out the phased remediation roadmap. Wherever the document says a control is implemented, it names the file and symbol at the implementation point so the reviewer can check it directly. Every negative statement traces back to the submitted repository. We do not claim a control that is not in the source.

1. Introduction

1.1 Purpose

This is the security documentation for **Dynamics Queue Admin** (`queadmin-frontend`), the dispatch-queue management console used by queue staff to manage driver queues, shipper requests, trips, queue organizations and real-time dispatch operations. It is submitted to the Information Network Security Agency (INSA) in support of the Secure Website Management Standard, 2014 EC. The document states the security requirements the asset has to meet, the controls actually implemented in the submitted release, the evidence for those controls, and, with equal prominence, the controls that are planned, outstanding, organizational, or owned by a separate system.

1.2 Scope

In scope: the client-side single-page application in this repository (React 19 + TypeScript + Vite 8), its production build, and its client-side security behaviours: session handling, route guards, form validation and sanitisation, storage handling, and error handling. The application is served at `queue.dynamicsroute.tech`.

Out of scope: the backend system of record (`transportBackEndNative`, Node.js/Express + MySQL), the driver mobile application, the shipper application, and the hosting platform itself. The backend is the enforcement boundary for authentication, authorization, session lifetime, upload handling and request validation; those obligations are identified here as dependencies and must be evidenced by the backend's own submission. The hosting platform (web server, TLS termination, certificate management, HTTP response headers) is a separate hosting engagement. **This repository contains no server configuration, no Content-Security-Policy and no security response headers; the document states this plainly rather than implying a server layer that does not exist in the source.**

1.3 Governing standard

The governing standard is the INSA *Secure Website Management Standard, 2014 EC*. It sets out five focus areas: Requirement Gathering and Analysis (2.1), Design (2.2), Implementation (2.3), Hosting (2.4), and Operation and Management (2.5), along with Annex A (eight common vulnerabilities and threats) and Annex B (a 102-item minimum security testing checklist). This document addresses all five focus areas and both annexes in turn.

1.4 Summary of outcome

The application layer is in reasonable shape. The console uses an httpOnly-cookie session model and never reads or stores a JWT in JavaScript (`getToken()` in `src/lib/auth.ts` returns `null` by design); login is one-time-password only, so there are no passwords to leak; forms are validated and sanitised with React Hook Form and Zod (`src/schemas/queue.ts` strips HTML tags and control characters); the client never renders untrusted HTML (a source scan found no `dangerouslySetInnerHTML`, `innerHTML`, `eval` or `new Function` in `src/`); logout clears the stored session profile and calls the backend logout; a global 401 handler ends the session on any unauthorized response; and production sourcemaps are disabled.

The largest gaps are real and are stated plainly. There is **no Content-Security-Policy and no security response header anywhere in the repository**, and no server configuration to deliver them, so anti-clickjacking, HSTS, anti-sniffing and related items cannot be evidenced from this asset. The error utility

`src/utils/parseError.ts` logs the **raw error object by default** (`console.error("[parseError]", error)`) and is not gated to development builds, so failure payloads can reach the browser console in production. The OTP is submitted to the backend as a JavaScript `Number`, which corrupts codes with a leading zero. The client also sends `roleId` and `statusId` on login and OTP verification even though the code comments and README claim it does not, so the backend must be confirmed to ignore them. There is no continuous integration, no dependency audit gate and no security linting plugin. Sections 14 and 16 quantify all of this.

2. Asset and contact information

2.1 Asset detail

Attribute	Value
Asset name	Dynamics Queue Admin (dispatch-queue management console)
Repository / asset code	`queadmin-frontend` — asset code `ASSET-DYN-QA-001`
Asset type	Web Application — client-side single-page application
Version	1.0.0 (`package.json` private version is `0.0.0`)
Technology	React 19, TypeScript, Vite 8, react-router-dom 7, Redux Toolkit / RTK Query, Zustand, axios, socket.io-client 4, react-hook-form + zod, i18next, date-fns, sonner
Production URL	`https://queue.dynamicsroute.tech/`
API endpoint	Same-origin `/api` by default in production; development API `https://dev.dynamicsroute.tech` (separate backend system)
Real-time channel	Socket.IO with `withCredentials` to the backend (`VITE_WEBSOCKET_URL` / API base)
Distribution	Static bundle (HTML/JS/CSS) served from a document root (`dist/`)

2.2 Classification

Attribute	Value
Asset classification	Business / internal-use web application; commercially sensitive
Data sensitivity	Confidential operational data and personal data (driver and shipper names, phone numbers, email, vehicle and queue records)
Availability class	Business-operational (daily driver queue and dispatch operations)
Owner	Marew Masresha, General Manager (business owner); Marew Masresha, General Manager (system/technical owner)
Criticality	Business-critical to daily operations. Unavailability or compromise of the console disrupts loading-place driver queueing and dispatch at every organization using it, and can delay freight movements. Recoverability and integrity of queue and journey records are important; the records themselves are held by the backend system of record.

2.3 Ownership and points of contact

Role	Name	Contact
Business owner	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`

Role	Name	Contact
Technical / system owner	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
Security contact	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
Hosting provider contact	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
Backend (API) team contact	Marew Masresha, General Manager	`mmarew@gmail.com` , `0983222221`
CERT / INSA reporting contact	Information Network Security Agency (INSA)	Via INSA reporting channel

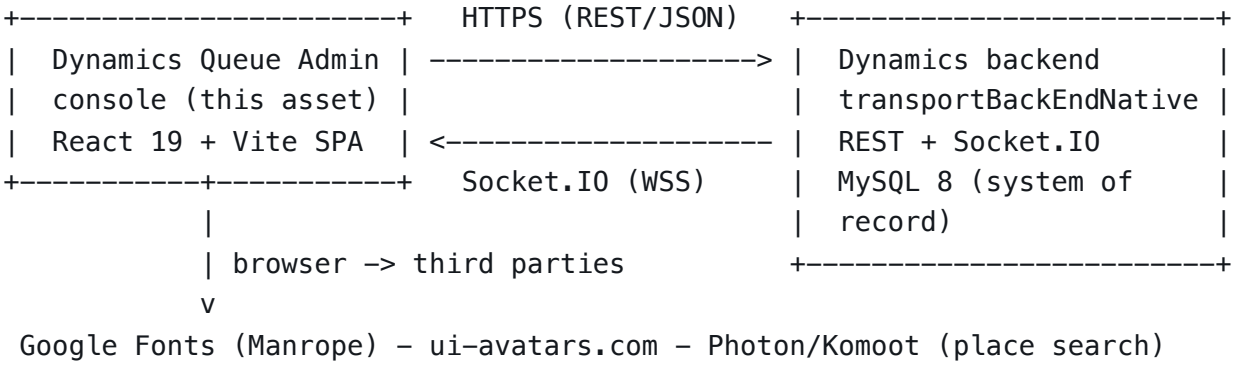
Contacts

Ownership and points of contact for Dynamics Transport are completed; the General Manager is the named business, technical, security and hosting contact. INSA reporting goes through the Agency's own channel, as shown above.

3. System description

3.1 Architecture

Dynamics Queue Admin is a standalone single-page application. It is loaded once from the static host, then communicates with a single backend over HTTPS (REST/JSON) and a single real-time channel (Socket.IO, with cookie credentials). It also calls third-party services directly from the browser for fonts, avatars and place search. The console is a queue-operations tool: queue organizations manage their driver queues, check drivers in, dispatch vehicles against shipper requests, and receive live queue updates. It holds no server-side logic and stores no database of its own; all state of record lives in the backend.



3.2 Security-relevant boundaries

Boundary	Description and control implication
Browser	The browser is the untrusted runtime, so the client-side checks are conveniences rather than security boundaries. This asset ships no CSP, so the browser never receives a script-execution policy to enforce.
SPA client	The console keeps only a non-sensitive user profile in `localStorage`. The session JWT lives solely in an httpOnly cookie set by the API host, and requests use `credentials: "include"` because the session rides a cookie, whether same-origin or cross-site.
Backend API	The **enforcement boundary for authorization and validation**. It derives identity and role from the session, applies the CSRF, session and rate-limit controls, and re-validates every input. None of that can be enforced from this asset; see §6 and §16.
Socket (Socket.IO)	Carries live queue events. It uses `withCredentials: true` and negotiates `websocket` with a `polling` fallback (`src/lib/socket.ts`). Authorizing individual messages stays with the backend.
Third-party services	Google Fonts, ui-avatars.com and Photon/Komoot place search are called directly from the browser. Each is a point where data, including search text, can leave the Dynamics trust boundary; see §10.

3.3 Data-flow summary

Flow	Transport	Notes
Authentication	HTTPS + OTP	A phone number is sent, the backend texts an OTP, the console verifies it (`POST /user/verifyUserByOTP`), and the backend sets the session cookie. Only the non-secret profile is stored locally. No password is involved.
Session bootstrap	HTTPS, cookie	The profile is rehydrated from `localStorage` on start so the UI can render immediately. The backend remains the authority, and any 401 on a later request clears the session and sends the user back to login.
Queue operations	HTTPS + cookie	REST/JSON for queue status, check-in, dispatch, shipper requests and organization management; the backend authorizes each one.
Real-time events	Socket.IO	Live queue and organization updates; the RTK Query cache is invalidated on receipt.
Place search	HTTPS to third party	Address and place text is sent to Photon/Komoot whenever a queue organization address is typed and does not match the local Ethiopian-city list. §10.

4. Security requirements and objectives

The standard's objectives are expressed through the CI4A+Accountability model (Confidentiality, Integrity, Authentication, Authorization, Availability, and Accountability). The table below states each objective for this asset, the assets and data it protects, and how the objective is met, including where it depends on the backend or the hosting platform.

Objective	Assets / data protected	How it is met
Confidentiality	Queue operations, driver and shipper personal data, session cookie	Production uses TLS. The session cookie is httpOnly, so no token is readable from JavaScript. Logout clears the local profile, the bundle carries no secrets, and production sourcemaps are switched off. Third-party place-search egress and raw error logging stay open (§9, §10).
Integrity	Queue entries, dispatch actions, shipper requests, organization records	Server-side validation and business rules authorize and persist every change; the client only presents data and forwards intent. Zod schemas on the client sanitise input before it leaves the browser.
Authentication	User accounts (phone identity)	A phone number plus a one-time password delivered by SMS. This is single factor, which is accepted for this population, and server-side OTP rate limiting remains a required backend control (§5).
Authorization	Queue-organization-scoped operations and administration	Role-based access is enforced by the backend, while a client route-boundary guard (`RoleGuard`) stops unauthorized UI from mounting. The backend stays the enforcement boundary (§6).
Availability	Queue and dispatch services during operating hours	The bundle is static and cacheable, and the socket client reconnects with a bounded number of attempts. The organization route check fails open on a transient error so a brief network blip does not lock operators out. That is a deliberate availability decision, noted in §6 and §16.
Accountability	Security events, changes, approvals	Application logging is aimed at development and, in one respect, is unsafe in production, because `parseError` logs raw errors by default. Server-side and organizational audit logging, change control, monitoring and risk assessment are outstanding (§13, §16).

> **Boundary discipline**
>
> Several of the objectives above can only be met in part from this repository, so the document names the system that owns each one rather than implying this asset controls it. In particular, authentication strength, session lifetime, rate limiting, CSRF defence, cookie flags and per-request authorization are all backend obligations.

5. Authentication and session management

5.1 Authentication flow

1. The user types a phone number and the console asks the backend for an OTP (`requestLoginOtp` , `POST /user/loginUser`).
2. A one-time password arrives by SMS. The console takes the six digits and verifies them (`verifyOtp` , `POST /user/verifyUserByOTP`).
3. On success the backend issues a session JWT in an **httpOnly cookie**. The console never reads or stores the token; the RTK Query `onQueryStarted` handler keeps only `userData` (`src/lib/redux/api/authEndpoints.ts`).
4. The console saves just the non-sensitive profile to `localStorage` under `queueadmin:user` , so the UI can show the user's name and role across reloads (`src/lib/auth.ts`).

> **Implemented application-layer controls (verified in source)**
>
> - **No JS-readable token.** `getToken()` returns `null` by design, and the record that is stored is `{ userData }` and nothing else. It is saved under `queueadmin:user` ; the older `queueadmin:auth` record, which could have held a token, is migrated and then deleted (`purgeLegacyAuth()`).
> - **Cookie-credential requests.** The axios layer (`src/lib/api.ts` , `withCredentials: true`) and the RTK Query base (`src/lib/redux/api/base.ts` , `credentials: "include"`) both send the session cookie. No `Authorization` header is ever built.
> - **Logout clears state.** `AuthContext.logout()` drops the socket, calls `POST /user/logout` on a best-effort basis to clear the httpOnly cookie server-side, removes the stored profile with `clearAuth()` , and dispatches the redux `logout` action (Annex B #39, #63).
> - **Central 401 handling.** Any 401 on any API call ends the session: the axios interceptor clears auth and redirects to `/login` , and `baseQueryWithReauth` dispatches `logout()` . A 401 during the socket handshake ends the session as well, so the client does not reconnect-loop against a dead cookie (`src/lib/socket.ts`).
> - **Phone masking.** The OTP screen shows a masked number (`maskedPhone` in `src/hooks/useOtpVerification.ts`).

Findings that prevent Pass status (verified in source)

- **OTP transmitted as a number.** `verifyOtp` sends `OTP: Number(OTP)` (`src/services/auth.service.ts`). A six-digit code that starts with a zero loses that zero in transit, so a valid code can be rejected, and the behaviour differs from the string-safe handling that was expected. Annex B #26 is marked **OPEN** on this ground.
- **roleId/statusId sent despite contrary comments.** Both `requestLoginOtp` and `verifyOtp` send `roleId: QUEUE_ORG_ADMIN_ROLE` (11) and `statusId: 1`, yet the in-file comment and the README say the client does *not* send a role. The backend has to be confirmed to take the role from the session and ignore these client-supplied values. Annex B #28, #36.
- **Raw error logging.** `parseError` logs the whole error object unless the caller passes `silent`, and it is not gated to development builds (`src/utils/parseError.ts:116`). This is discussed in §9, and it is why the token and log-disclosure items are marked **OPEN**.
- **Server messages surfaced.** `resolveServerMsg` returns the backend's `message` / `error` text when it is present, so a failed OTP can display backend prose that tells "unknown user" apart from "wrong code", which is username enumeration. Annex B #35 is **OPEN**.

5.2 Session lifetime

Session lifetime is **set by the backend and not visible to the client**. `VITE_JWT_EXPIRATION_DAYS=7` appears in `.env.development` and `.env.example`, but nothing in `src/` reads it; it is misleading and should be deleted. The backend session policy documentation (README, referring to `Utils/SessionPolicy.js`) makes lifetimes role-dependent: **Admin (3) and SuperAdmin (6) = 24 hours; Queue Org Admin (11) and Queue Dispatcher (12) = 365 days.** The 365-day lifetime for queue staff is unusually long and appears as a hardening recommendation in §17. The periodic-expiration requirement (Annex B #76) therefore rests entirely on the backend.

5.3 Residual items to confirm with the backend team

These cannot be implemented in this repository

Item	Why it matters
Cookie flags: `HttpOnly; Secure; SameSite`; `Path=/`	The production build defaults to a same-origin `/api` path, but if the SPA and API are split across hosts, the cookie needs `SameSite=None; Secure`. **Confirm what the backend actually sets.** Annex B #67–#70.
Server-side OTP rate limiting and lockout	The console only enforces a 60-second resend cooldown (`RESEND_COOLDOWN_SECONDS = 60`). Without a server-side counter, OTPs can be guessed and SMS-toll abuse becomes possible. Annex B #37.
Server-side session revocation and regeneration	Regenerating the session on login and revoking it on logout or credential change both have to happen on the server. Annex B #64–#66, #73.
Periodic session expiration and idle timeout	Maximum session age and idle timeout are backend decisions, and the 365-day window needs review. Annex B #76.
Session identifier quality	Unpredictability, collision resistance and TLS-only transmission are properties the backend has to provide. Annex B #71, #72, #74, #75.
JWT returned in the response body	`VerifyOtpResponse` declares a `token: string` field (`src/types/queue.ts`). If the cookie model is the intended design, the body token should be dropped or at least shortened. Annex B #36, #74.

6. Authorization and access control

6.1 Role model

The console serves a small set of queue-console roles: Queue Org Admin (role 11), Queue Dispatcher (role 12), platform Admin (role 3) and Super Admin (role 6). The role is stored in the profile and resolved by the backend at login. Being a member of a particular queue organization is a backend relationship, and the client cannot grant it.

6.2 Client route-boundary guards

Two components wrap the protected routes and run *before* the child renders:

- `src/components/auth/RoleGuard.tsx` checks that the user holds one of the allowed role IDs (`{ 11, 12, 3, 6 }`); if not, the guard sends them to `/login`. Roles 3 and 6 are in the list because they are legitimate platform operators of the queue console.
- `src/components/auth/ProtectedRoute.tsx` checks that the user is authenticated and holds at least one queue organization; if not, they are sent to `/setup-org`. It waits for the organization query to settle before deciding.

> **Fail-open behaviour in ProtectedRoute (Annex B #32)**
>
> When the organization query errors, `ProtectedRoute` returns `true` for `hasOrg` and lets the user through: `orgsError ? true : false` (`src/components/auth/ProtectedRoute.tsx:48–52`). That was a deliberate availability choice, because a brief backend failure should not lock operators out of a live queue, but it is a fail-open condition and is marked **OPEN** under Annex B #32. It does not bypass authentication (the session check still runs first) or backend authorization, and it only affects the client's organization-setup gate. The fix is to tell an authorization failure (redirect) apart from a transport failure (show a retryable error, not the app).

> **The backend is the enforcement boundary, stated explicitly**
>
> The route guards are a **UI safety control, not an authorization boundary**. A user who forges their stored profile or calls the API directly still has to be denied by the backend, which must:
>
> - take `roleId` / `statusId` from the session and never from the request body (the client sends them for legacy reasons, see §5.1);
> - scope every queue-organization query and mutation to the cookie identity;
> - authorize every organization and queue identifier against the session's memberships.
>
> Until the backend team shows this, Annex B #23, #24, #27 and #28 stay open.

6.3 Per-action checks

Each action exposes only what the role permits, so for example bid approval appears just to role 11 in the bidding board. Every mutating request goes to the backend, and the backend alone accepts or rejects it. No client-side decision is treated as final.

6.4 Privilege categories and lifecycle

The code implements privilege categorization (INSA 2.5.D) through the role set above and the queue-organization membership model. Granting and revoking access, and rotating credentials on a fixed schedule, are organizational procedures that are not evidenced in this repository (INSA 2.5.C and 2.5.E).

7. Transport security

7.1 HTTPS

The production URL is HTTPS-only (`queue.dynamicsroute.tech`). In development the application is set to call the API over `https://` (`VITE_API_BASE_URL=https://dev.dynamicsroute.tech`), and a production build defaults to the same-origin `/api` path, which inherits the page's TLS. The socket client targets the same host and sends the cookie with `withCredentials: true`.

> **Not independently verified for this submission**
>
> This repository contains no server configuration, so the HTTP-to-HTTPS redirect, the TLS protocol and cipher set, and the certificate chain on the production host were **not independently assessed** here. The application itself only ever addresses HTTPS origins in production. Annex B #13 and #15 are marked PASS on the basis of the application's own transport choices (HTTPS/WSS and no mixed content); #14 is marked PASS with the platform redirect noted as unverified; #16 (HSTS) and #17–#18 (TLS/certificate) are OPEN.

7.2 Socket transport

`src/lib/socket.ts` initializes Socket.IO with `transports: ["websocket", "polling"]`, `withCredentials: true`, `reconnectionAttempts: 5`, `reconnectionDelay: 1500`, `reconnectionDelayMax: 5000` and `timeout: 20000`. The polling fallback is kept for restrictive networks. Unlike a client pinned to WebSocket only, the polling handshake is an ordinary HTTP request, so it also has to be protected by the platform's cookie and CSRF controls.

7.3 HSTS and cookie scope

No `Strict-Transport-Security` header is defined anywhere in the repository, so HSTS is **absent** (Annex B #16). Because a production build defaults to a same-origin API path, the session cookie may be same-site. If the deployment instead splits the SPA and API across registrable domains, the cookie has to be `SameSite=None; Secure` and CSRF defence cannot rely on the default `SameSite=Lax` (see §5.3 and §14 #61, #62). The deployment and the backend have to settle this before submission.

8. Application security controls

8.1 Content-Security-Policy

> **There is no CSP, stated plainly**
>
> A scan of the whole repository (`src/`, `public/`, `index.html`, `vite.config.ts`) found **no Content-Security-Policy**: no meta tag, no build-time injection, and no server configuration to deliver a header. This asset sends the browser no script-execution policy at all. Annex B #83 (non-specific content types), #85 (anti-sniffing) and #96 (anti-clickjacking) are therefore OPEN, and there is no defence-in-depth layer behind React's output encoding for Annex B #43.

One thing to account for later: `index.html` embeds an inline `<script type="application/ld+json">` structured-data block. A strict `script-src` policy would need to accommodate it with a hash or nonce, or the block would have to move to an external file. That is recorded as a dependency of the remediation in §17.

8.2 Security response headers

This repository configures no security response headers: `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Opener-Policy` and `Cross-Origin-Resource-Policy` are all missing. The hosting platform may add some of them, but that sits outside this asset and is **not evidenced**. They are recorded as hosting items in §16.

8.3 Client-side output encoding

- **React escaping.** Every untrusted value goes through JSX text nodes and attribute bindings, which React escapes by default (Annex B #43).
- **No raw HTML sinks.** A source scan found no `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function` or `document.write` in `src/`.
- **Input sanitisation.** Zod transforms strip HTML tags and control characters from names, addresses and emails before they are stored or sent (`src/schemas/queue.ts`: `stripTags`, `stripCtrl`, `sanitizeText`, `sanitizeEmail`).

Without a CSP these controls are the only thing standing between script injection and the browser, which is why §17 asks for a policy as the first server-side change.

8.4 Build hardening

- **Sourcemap suppression.** `vite.config.ts` sets `build.sourcemap = process.env.NODE_ENV !== "production"`, so development builds carry sourcemaps and production builds do not. The effect is right, but it is implicit; pinning an explicit `false` for production is recommended (Annex B #6).
- **No secrets in the bundle.** Only public configuration (API and socket hosts) is pulled in from environment variables. No API keys or credentials are embedded or committed.

9. Input validation and data handling

9.1 Form validation and sanitisers

Forms use `react-hook-form` with `zod` schemas (`src/schemas/queue.ts`) for login, registration, organization setup, check-in and dispatch. The schemas constrain length and format, and they transform the values: strip HTML tags (`stripTags`), drop non-printable control characters (`stripCtrl`), collapse whitespace (`sanitizeText`) and normalise email (`sanitizeEmail`). Phone numbers are normalised to E.164 (`+251…`) by `src/utils/phoneFormatter.ts` and `src/lib/authConfig.ts`. Client-side validation is a usability and hygiene control only, and the backend has to re-validate every field.

9.2 File uploads

> **No file upload feature**
>
> The console neither uploads nor downloads files: a source scan found no `FormData`, no file input, no `FileReader` and no multipart request. The Annex B File Upload group (#78–#82) is therefore marked **N/A** for this asset. If uploads are added later, for example for compliance documents, the validation and storage controls have to be designed and added to this document.

9.3 Production error handling and log hygiene

> **Finding: raw error logging is not dev-gated**
>
> `src/utils/parseError.ts` runs `console.error("[parseError]", error)` unless the caller passes `{ silent: true }`, and it is **not** gated behind `import.meta.env.DEV`. The logged error object can include the request configuration and response data, which for auth calls can carry the phone number and OTP. That is a production information-disclosure and log-hygiene gap. It is why Annex B #4 (sensitive information in logs/errors) and #77 (disclosure of tokens in logs) are marked **OPEN**, and it also contributes to #7.

The path shown to the user is better: `parseError` returns mapped, generic messages per HTTP status (401 → "Session expired", 403 → "You do not have permission", and so on). But when the backend sends a `message` / `error` string it is passed through to the user (`resolveServerMsg`), which can reveal internal detail. Restricting this to an allow-list of server messages would close it.

9.4 Client-side data and PII handling

The session JWT lives only in the httpOnly cookie, while `localStorage` holds the non-secret profile (`queueadmin:user`: name, phone, email, role) used for rendering. Logout calls `clearAuth()`, which removes that profile from both `localStorage` and `sessionStorage`. Other local keys (`app_language`, `queueadmin:store`, a UI selection only, `insa_security_checklist_v1`, theme and sidebar preferences) hold no personal data and are left alone; that is acceptable, though it is noted here for completeness. The remaining risk is that the profile sits in cleartext while the user is signed in. That was a deliberate routing decision and it is recorded in §16.

10. Third-party and integration security

The console calls third parties straight from the browser. Each one is listed below with its real call site, and no other external origin appears in the source.

Third party	Purpose	Security notes
Google Fonts (`fonts.googleapis.com`)	Manrope typeface	Loaded with an `@import` at the top of `src/index.css`. Only static assets are fetched, and no application data is sent, but it does add an external dependency to the critical rendering path and a request to a third party on every page load.
ui-avatars.com	Generated avatar images	Called from `DashboardLayout.tsx:42`, `DesktopProfileSection.tsx:44` and `MobileProfileView.tsx:41`. The user's display name goes into the query string (`?name=…`), so part of the personal data is handed to the third party and shows up in its access logs. Hosting avatar generation locally would remove that egress.
Photon / Komoot (`photon.komoot.io`)	Address / place search autocomplete	**PII egress.** `src/hooks/usePhotonSearch.ts` sends the place or address text the user typed to Photon whenever it does not match the local Ethiopian-city list, debounced at 350 ms. Queue-organization addresses and search terms therefore leave the Dynamics boundary. The base is configurable via `VITE_PHOTON_URL`, and the fix is a backend proxy or a self-hosted instance (§17).

> **No CSP restricts these origins**
>
> Because the asset has no Content-Security-Policy, the browser is free to contact any origin. A policy that allow-lists only these three origins, plus the backend, would put the third-party surface in writing and enforce it mechanically. Until that exists, Annex B #8 and #11 stay **OPEN**.

11. Dependency and supply-chain management

11.1 Dependency auditing

`npm audit --omit=dev` reports **one high-severity vulnerable runtime dependency**. The fix is a single command (`npm audit fix`), but it is **not yet applied** and nothing enforces it. That fails Annex B #90 (missing security updates).

Package	Range	Severity	Notable advisories
`axios`	1.0.0 – 1.19.0	High	Prototype-pollution gadgets (fetch adapter, default instance, toFormData options), ReDoS in the `data:` URL parser, redirect-based SSRF, HTTP/2 adapter DNS/proxy bypass, and header injection through inherited headers (12+ advisories)

11.2 Static analysis and security linting

The ESLint configuration (`eslint.config.mjs`) pulls in `@eslint/js`, `typescript-eslint`, `eslint-plugin-react` and `eslint-plugin-react-hooks`, and nothing else. There is **no security plugin** (no `eslint-plugin-security`), so risky patterns such as non-literal regular expressions, timing comparisons or unsafe object injection never get flagged. `oxlint` is installed and configured, but it is not a security gate. The `lint` script runs `eslint . --ext ts,tsx --max-warnings 0`, yet the README describes `npm run lint` as oxlint, so the documentation and the script disagree.

11.3 A gap: unused and mis-declared dependencies

- `@tanstack/react-query` is listed in `package.json` but **nothing imports it**; the project uses RTK Query. It is dead weight that widens the attack surface and the audit scope for no benefit.
- `.npmrc` sets `legacy-peer-deps=true`, which suppresses peer dependency resolution and makes the install graph harder to reproduce.

11.4 Continuous integration

> **No CI today**
>
> There is no `.github/` directory, no Dependabot configuration, no secret scanning and no build pipeline. Nothing catches dependency drift, checks that a rebuild stays clean on deploy, or runs the test suite (107 tests, all passing) automatically. A suggested workflow is in §17.

12. Deployment and hosting security

12.1 Deployment model

The console is built to a static bundle (`npm run build`: `tsc -b && vite build`) and served from a document root (`dist/`). The production host serves that static site and terminates TLS. In development the server proxies `/api` and `/socket.io` to the backend (`vite.config.ts`).

12.2 Server configuration status

This repository contains no server configuration at all: no nginx file, no header set, no 404 blocks, no caching policy. The table below therefore lists the hosting-layer items as unverified rather than describing a configuration we have proposed. The hosting engagement has to complete and evidence them.

Item	Status	Notes
Security response headers	ABSENT / UNVERIFIED	No header set exists in the repository, and the platform behaviour is unknown.
Content-Security-Policy	ABSENT	No meta tag, no build injection, no header.
Extraneous-file 404 blocks	UNVERIFIED	No guard is defined for `/.env`, `/.git` or `*.map`.
Cache-Control policy	UNVERIFIED	HTML should be `no-store` and hashed assets `immutable`.
HTTP→HTTPS redirect	UNVERIFIED	Assumed to sit with the platform; not evidenced here.
TLS termination	UNVERIFIED	The platform manages certificates, and the configuration was not independently assessed.
Staging environment	ABSENT	Patch and config changes cannot be tried on a separate environment (INSA 2.5.M).

12.3 Organizational hosting items

The standard's hosting focus area (2.4) is mostly organizational, and repository artifacts cannot satisfy it. It asks for a hosting security requirements document, an assessment of the provider's technical and non-technical capability, the hosting location, and a written security agreement. All of these are recorded as outstanding in §16.

13. Logging, monitoring and incident response

13.1 Current state

- Application logging is aimed at development and is **not safe in production** in one respect: `parseError` logs the raw error object by default and is not dev-gated. The socket client also logs connection errors through `console.warn`, including the underlying error object (`src/lib/socket.ts`).
- Nothing on the client transmits security events, no server-side audit log is evidenced for this asset, and there is no alerting.
- With no CSP in place, there is no way to report violations (`report-uri` / `report-to`).

13.2 Gaps

Gap	INSA reference
No breach or misuse monitoring for the web application	2.5.K (current status: Fail)
No access/audit logging of security-relevant events	2.5.B, 2.5.K
No periodic security testing or auditing programme	2.5.G, 2.5.H
No documented incident response or breach-notification procedure	2.4.E.ii, 2.5.A.ii

13.3 Recommendations

- Put `parseError`'s raw logging behind `import.meta.env.DEV` and log only a terse, payload-free object in production. This is a quick code-only change.
- Turn on structured application monitoring and attack or misuse detection for the public host.
- Add a CSP report endpoint once a policy is deployed, and triage what comes back.
- Write down a breach-handling and emergency-communication procedure, with named owners for each step.
- Set up the CERT/INSA reporting contact and the escalation path. Reporting goes to the Information Network Security Agency (INSA) through its published channel, with the General Manager as the named internal escalation point.

14. Testing and assurance

14.1 Automated tests

The repository has a Vitest suite that a reviewer can **run independently**: the latest run reports **107 passing tests across 7 files** (`npm test`, executed while this document was being prepared). The suite covers a few security-relevant areas:

Test file	Tests	Coverage
`src/__tests__/insaStandard.test.ts`	6	Checks the structure of the embedded INSA model: 5 focus areas, 35 requirements, 102 Annex B tests, 8 Annex A vulnerabilities.
`src/__tests__/securityChecklist.test.ts`	14	Checks the whitelisting and sanitisation of the checklist state kept in localStorage; invalid, oversized and wrongly typed entries are dropped.
`src/__tests__/queueMutations.test.ts`	40	Covers queue business logic and mutation validation, including `parseError` behaviour and status-transition rules.
`src/__tests__/journeyStatus.test.ts`	19	Journey and order status mapping.
`src/__tests__/phoneFormatter.test.ts`	12	Phone normalisation and masking.
`src/__tests__/ordersLocation.test.ts`	12	Location parsing and validation.
`src/__tests__/ordersDataMapper.test.ts`	4	Order payload mapping.

The suite is **not run in CI**, because there is no CI. There are also no automated tests for the auth flow, the route guards or socket handling.

> **How to read the Annex B table**
>
> PASS means implemented and verified in the submitted source, with a specific file named. OPEN means failing, absent, dependent on the separate backend, or organizational with no evidence. N/A means genuinely not applicable to this asset: a static SPA has no server-side XML parser and executes no uploaded file, and this console performs no file uploads. Server and hosting rows are left OPEN because **no server configuration exists in the repository** to verify them against.

14.2 Annex B — minimum security testing checklist (102 items)

No	Test	Status
Information Disclosure		
1	Extraneous files in the document root. No server configuration or 404 guard covers `/.env`, `/.git` or `*.map`, and live host behaviour is not evidenced.	OPEN

No	Test	Status
2	Extraneous directory listings. No server configuration exists to confirm autoindex is disabled.	OPEN
3	Accessible debug functionality. The application has no debug pages, but server-side debug endpoints are not evidenced.	OPEN
4	Sensitive information in log and error messages. `parseError` logs the raw error object by default and is not dev-gated.	OPEN
5	Sensitive information in `robots.txt`. It holds only `Allow: /` and the sitemap URL, nothing sensitive.	PASS
6	Sensitive information in source code. No secrets are in the bundle, and production sourcemaps are disabled.	PASS
7	Disclosure of internal addresses. No internal addresses appear in the bundle. Residual disclosure through raw server messages is noted in §9.	PASS
Privacy and Confidentiality		
8	Sensitive information stored in URLs. Place-search text is sent to Photon/Komoot, no `Referrer-Policy` exists, and avatar names appear in ui-avatars URLs.	OPEN
9	Unencrypted sensitive information stored at the client-side. Logout purges the stored profile (name, phone, email) from both storages, and the remaining local keys hold no personal data.	PASS
10	Sensitive information stored in externally archived pages. No `Cache-Control` policy is defined or verified.	OPEN
11	Content included from untrusted sources. Google Fonts, ui-avatars.com and Photon/Komoot are all pulled in, and there is no CSP to constrain that egress.	OPEN
12	Caching of pages with sensitive information. No `Cache-Control` configuration exists in the repository.	OPEN
13	Insecure transmission of sensitive information. The production URL and API are HTTPS, and the socket uses WSS/HTTPS with cookie credentials.	PASS
14	Non-SSL/TLS pages on sites processing sensitive information. The application only ever addresses HTTPS origins in production; the host redirect is not independently evidenced.	PASS
15	SSL/TLS pages served with mixed content. All fetched resources (fonts, avatars, API, socket) are HTTPS, so there is no mixed content.	PASS
16	Missing HSTS header on full SSL sites. No HSTS is defined anywhere in the repository.	OPEN
17	Known vulnerabilities in SSL/TLS. The platform TLS configuration is not independently assessed.	OPEN
18	Weak, untrusted or expired SSL certificates. The platform certificate is not independently assessed.	OPEN
19	Use of unproven cryptographic primitives. The SPA performs no custom cryptography and relies on browser TLS.	N/A

No	Test	Status
20	Incorrect usage of cryptographic primitives. There is no custom cryptographic implementation in the asset.	N/A
State Management		
21	Client-side state management. The non-secret profile sits in `localStorage` alongside smaller UI keys. It is not encrypted while signed in, which server-side authority and the logout purge partly offset.	OPEN
22	Invalid state transitions. The business state machine is enforced by the backend, not this asset.	OPEN
Authentication and Authorization Process		
23	Missing authentication or authorization. Route-boundary guards exist, the backend must still enforce per request, and `ProtectedRoute` has one fail-open branch.	OPEN
24	Client-side authentication. `isAuthenticated` and the role are derived from the locally stored profile; the backend remains the boundary.	OPEN
25	Predictable and default credentials. Login is OTP-only, so there are no default passwords; backend assurance of OTP strength and delivery is required.	OPEN
26	Predictable authentication or authorization tokens. The OTP is cast to `Number` before transmission, which corrupts codes with a leading zero (`src/services/auth.service.ts`).	OPEN
27	Authentication or authorization based on obscurity. Role and membership are enforced by the backend; the client guard is UI only.	OPEN
28	Identifier-based authorization and privilege escalation. The client sends `roleId` (11) and `statusId` (1) on login and OTP even though comments claim otherwise, so every organization and queue identifier must be authorized server-side.	OPEN
29	Acceptance of weak passwords. No password authentication is used (phone + OTP).	N/A
30	Account recovery process. Account recovery, if any, is owned by the backend.	OPEN
31	Any “remember me” function. The console has no such function.	N/A
32	Fail-open conditions. `ProtectedRoute` lets the user through when the organization query errors.	OPEN
33	Impersonation function or reCAPTCHA. There is no impersonation function, and CAPTCHA is a backend decision.	OPEN
34	Plaintext retrieval of passwords. No passwords are stored or retrieved.	N/A
35	Username enumeration and uniqueness. Raw backend `message` / `error` text is shown to the user when present (`resolveServerMsg`), which lets OTP and unknown-user responses be told apart.	OPEN
36	Unsafe distribution of credentials. `VerifyOtpResponse` declares a `token` field returned in the response body, and the backend should review that.	OPEN

No	Test	Status
37	Missing rate limiting on authentication functionality. The client has only a 60 s resend cooldown, so server-side limiting is required.	OPEN
38	Missing re-authentication when changing credentials. This is backend-owned.	OPEN
39	Missing logout functionality. Logout disconnects the socket, calls the backend logout, clears both storages and dispatches the redux logout.	PASS
40	Multi-stage mechanisms. Multi-stage flows are backend-owned.	OPEN
User Input Management		
41	SQL injection. There is no SQL in this asset; it is a backend responsibility.	OPEN
42	Path traversal and filename injection. The console has no file upload and accepts no filesystem path input, so there is no traversal surface.	N/A
43	Cross-site scripting (stored, DOM and reflected). React output encoding applies, a source scan found no `dangerouslySetInnerHTML` or `innerHTML`, and the Zod sanitisers strip tags and control characters. No CSP defence-in-depth exists, which is noted above.	PASS
44	System command injection. A browser SPA executes no system commands.	N/A
45	XML injection. This asset does no XML parsing.	N/A
46	XPath injection. There is no XPath processing.	N/A
47	XSL(T) injection. There is no XSLT processing.	N/A
48	SMTP injection. This asset sends no mail.	N/A
49	SSI injection. This asset has no server-side includes.	N/A
50	HTTP header injection. Response headers are server-controlled, which is a backend responsibility.	OPEN
51	HTTP parameter injection. The client passes parameters to the backend, so the server must validate them.	OPEN
52	Native software flaws (buffer overflow, integer bugs, format strings). The runtime is managed JavaScript and this asset contains no native code.	N/A
53	LDAP injection. There is no LDAP processing.	N/A
54	Dynamic scripting injection. A source scan found no `eval`, `new Function` or `document.write`, so no dynamic-script sink exists. This is held by review rather than by a CSP.	PASS
55	Regular expression injection. No security lint gate flags non-literal regex constructions, and this was not assessed exhaustively.	OPEN
56	Data property and field injection. The wide request typings invite mass assignment, so a server-side allow-list is required.	OPEN
57	Protocol-specific injection. There is no custom protocol handling.	N/A

No	Test	Status
58	Expression language injection. This asset uses no expression language.	N/A
59	Fuzzing of request parameters. No fuzzing was performed; the backend owns request parsing.	OPEN
60	Arbitrary redirection. The only programmatic redirect target is the constant `/login`, and the only dynamic `href` is a `tel:` URI, so no user-controlled redirect exists.	PASS
Session Management		
61	Cross-site request forgery (CSRF). The session is cookie-based, and backend CSRF defence is not evidenced.	OPEN
62	Predictable CSRF tokens. The client holds no CSRF token, which is a backend responsibility.	OPEN
63	Missing session revocation on logout. `clearAuth()` runs, along with a best-effort server logout and a full storage purge; server revocation must be confirmed.	PASS
64	Missing session regeneration on login. Backend-owned.	OPEN
65	Missing session regeneration when changing credentials. Backend-owned.	OPEN
66	Missing revocation of other sessions when changing credentials. Backend-owned.	OPEN
67	Missing `Secure` flag on session cookies. Backend configuration is unverified.	OPEN
68	Missing `HttpOnly` flag on session cookies. The design is httpOnly, but the backend configuration is unverified; the client never reads a token.	OPEN
69	Non-restrictive domain on session cookies. The same-origin default versus a cross-site deployment is still ambiguous, and this is backend-owned.	OPEN
70	Non-restrictive or missing path on session cookies. Backend-owned.	OPEN
71	Predictable session identifiers. Backend-owned.	OPEN
72	Session identifier collisions. Backend-owned.	OPEN
73	Session fixation. Backend-owned.	OPEN
74	Insecure transmission of session identifiers. The cookie rides TLS alone, but the JWT is also returned in the response body.	OPEN
75	External session hijacking. This is a backend and transport property, and the cookie flags are still to be confirmed.	OPEN
76	Missing periodic expiration of sessions. The client env variable is dead, and lifetimes are backend-only at 24 h (roles 3/6) and 365 d (roles 11/12).	OPEN
77	Disclosure of tokens in logs. `parseError` logs the raw error object by default, and request or response payloads for auth calls can carry the phone number and OTP.	OPEN
File Upload		
78	Storage of uploaded files in the document root. This asset has no upload feature.	N/A

No	Test	Status
79	Execution or interpretation of uploaded files. The console executes no uploaded file.	N/A
80	Uploading outside the designated upload directory. No upload feature exists.	N/A
81	Missing size restrictions on uploaded files. No upload feature exists.	N/A
82	Missing type validation on uploaded files. No upload feature exists.	N/A
Additional Checks		
83	Missing or non-specific content type definitions. No server or content-type configuration exists.	OPEN
84	Missing character set definitions. `index.html` declares `<meta charset="UTF-8">`.	PASS
85	Missing anti content sniffing measures. There is no `X-Content-Type-Options: nosniff` anywhere.	OPEN
86	XML external entity expansion. This asset does no XML processing.	N/A
87	External DTD parsing. There is no DTD processing.	N/A
88	Extraneous or dangerous XML extensions. There is no XML subsystem.	N/A
89	Recursive entity expansion. There is no XML entity processing.	N/A
90	Missing security updates. `npm audit --omit=dev` reports a high-severity advisory for `axios`, which is not yet fixed and is not gated.	OPEN
91	Unsupported or end-of-life software versions. No automated dependency gate exists.	OPEN
92	HTTP TRACK and TRACE methods and other dangerous methods. No server configuration restricts methods, and live behaviour is not evidenced.	OPEN
93	Extraneous functionality. No debug endpoints or dead application surfaces turned up in `src/`, but server identity disclosure is unverified.	OPEN
94	Default credentials. Login is OTP-only, and default-credential management sits with the backend and hosting.	OPEN
95	Bugs in web server software. The web server version was not assessed.	OPEN
96	Missing anti-clickjacking measures. There is no `frame-ancestors` and no `X-Frame-Options`, and a meta tag cannot deliver either.	OPEN
97	Open redirection. No user-controlled redirect exists; only the constant `/login` target and `tel:` links.	PASS
98	Insecure cross-domain access policy. No `Permissions-Policy`, COOP or CORP headers exist.	OPEN
99	Missing rate limiting on e-mail functionality. E-mail is backend-owned, and this asset sends none.	OPEN
100	Missing rate limiting on resource-intensive functionality. Backend-owned.	OPEN

No	Test	Status
101	Inappropriate rate limiting that results in a denial of service. This is backend dimensioning, and it was not assessed.	OPEN
102	Application- or setup-specific problems. No app-specific defects were found in this code review, and continuous testing is recommended.	OPEN

14.3 Checklist result summary

Pass	Open	N/A	Total
14	63	25	102

The fourteen passing items are verified in the submitted source: #5, #6, #7, #9, #13, #14, #15, #39, #43, #54, #60, #63, #84 and #97. Most of the 63 open items are backend, hosting or organizational obligations that this repository cannot close on its own. The client-side gaps that *can* be closed here are #4 and #77 (dev-gate raw error logging), #26 (send the OTP as a string), #28, #35 and #36 (stop sending roleId/statusId; stop surfacing raw server messages), and #55 (add a security lint gate); then #90 (apply `npm audit fix`).

15. Annex A — Common vulnerabilities and threats

Annex A lists eight common website vulnerabilities and threats that have to be considered during design, implementation, hosting, operation and management. The table says how the console is affected and what control is applied, including who that control belongs to.

Vulnerability / threat	Console exposure	Control applied
1. SQL Injection	The SPA issues no SQL. All data access sits behind the backend API.	A backend responsibility, and parameterised queries belong there. The client only sends structured JSON over TLS. Marked **OPEN** pending backend evidence.
2. Unchecked Path Parameter / Directory Traversal	This console takes no filesystem path input, does no file upload and never opens a document supplied by the server.	There is no traversal surface in this asset. Server document serving and confinement of the upload directory are backend and hosting responsibilities. **N/A** for the console.
3. Improper Session Management	Session cookie, an optimistic local profile, and a role-dependent lifetime.	Handled client-side: an httpOnly-cookie model where the token is never JS-readable, a central logout, a full storage purge on logout, 401-driven session end and socket 401 handling. The backend still has to confirm the cookie flags, periodic expiry, regeneration and revocation, and the 365-day lifetime for roles 11/12 should be reviewed. Partially **OPEN**.
4. Cross-Site Scripting	Any untrusted string rendered into the DOM, server-supplied messages, OTP errors.	React's output encoding, no raw HTML sinks (`dangerouslySetInnerHTML` / `innerHTML` absent) and Zod sanitisers on form input. **No CSP exists**, so there is no script-execution policy as defence-in-depth. Marked **PASS** at the application layer, with the CSP gap recorded in §17.
5. CSRF	State-changing REST calls authenticated by a cookie session.	Not controllable from this asset. The backend must implement CSRF defence (token or Origin checking) and set the cookie `SameSite` attribute correctly for its deployment topology. Marked **OPEN**.
6. HTTP Header Injection	Response headers, and any reflected input that reaches them.	The SPA constructs no response headers. This is a backend and hosting responsibility, and no server configuration exists in the repository. Marked **OPEN**.
7. Mail Header Injection	The console sends no e-mail. OTP SMS delivery is backend-owned.	Not applicable to this asset; any mail or SMS composition is a backend responsibility. **N/A** for the console, with backend assurance required.
8. Lack of Authentication and Authorization	Route-level UI access, and every API call.	Implemented client-side: OTP authentication, a `RoleGuard` role boundary and a `ProtectedRoute` organization gate, with one fail-open branch. The **backend remains the enforcement boundary** and must derive identity and role from the session and authorize every resource. Partially **OPEN** pending backend evidence.

16. Compliance summary and deviations register

16.1 Focus-area status

The status below reflects what is in this repository and what can be verified from it. It is deliberately blunt: controls the application actually implements are kept apart from organizational, hosting and backend items that are outstanding or unknown. The five focus areas map to the standard's sections 2.1–2.5 and are also modelled inside the application itself in `src/data/insaStandard.ts` (35 requirements).

2.1 Requirement Gathering and Analysis

Req	Status	Notes
A–F	PARTIAL	Business requirements, security objectives and interfaces are implemented and documented in the SDD/TDD artefacts and the embedded standard model. Formal stakeholder sign-off evidence is not present.
G	UNKNOWN	Developer contracts covering source-code intellectual property are not in evidence. This is organizational.
H	UNKNOWN	Top-management approval of the security requirements is not documented.

2.2 Design

Req	Status	Notes
A	PARTIAL	Security is designed into the asset (httpOnly-cookie session, role guards, Zod sanitisers, storage discipline) and documented in the SDD/TDD artefacts, but the design lacks a CSP/header strategy and no mapping to the Secure Software Development and Management Standard V.1 is evidenced.
B	UNKNOWN	Design review and approval by the responsible body is not documented.

2.3 Implementation

Req	Status	Notes
A	PARTIAL	Secure software standard not mapped; CI and a security lint baseline are outstanding.
B	UNKNOWN	No evidence of developer security training.
C	UNKNOWN	CMCSRS accreditation before hosting. This is organizational.
D	UNKNOWN	Pre-hosting audit, also organizational.
E	PARTIAL	This repository contains no secrets; repository access control is not verified.

2.4 Hosting

Req	Status	Notes
A–C	UNKNOWN	No hosting security requirements document, and no assessment of the provider's technical or non-technical capability.
D	UNKNOWN	Hosting location unverified (relevant if the website is a government asset).
E	UNKNOWN	No full security agreement; the twelve named provisions are still outstanding.

2.5 Operation and Management

Req	Status	Notes
A	UNKNOWN	No website management policy (administrative responsibilities, emergency communications, content management).
B–D	PARTIAL	`RoleGuard` implements privilege categorization (D) in code; documented operating procedures are unknown.
E	PARTIAL	Session revokes on logout; credential rotation interval unknown.
F	FAIL	No quarterly vulnerability and risk assessment evidence.
G–H	UNKNOWN	No periodic testing/auditing evidence.
I	PARTIAL	No CI or formal change-control procedure; changes are committed but not approved by a security owner.
J	UNKNOWN	No in-advance operational risk assessment for significant changes.
K	FAIL	No breach or misuse monitoring.
L–M	PARTIAL	Dependency remediation (`axios`) pending; no documented patch plan and no staging environment.
N	PARTIAL	The application uses HTTPS in production; certificate renewal and HSTS are not evidenced.
O	UNKNOWN	No monitoring of the provider's security updates.

> **Assessment**
>
> The application-layer controls are in reasonable shape. The compliance scaffolding is largely absent, and the server layer provides no security headers at all. The biggest gaps are the CSP and header layer (hosting), the raw error logging, the OTP and role findings, and continuous integration.

16.2 Deviations register

ID	Deviation	Justification / owner
D1	Backend controls (CSRF, cookie flags, OTP rate limiting, session regeneration and revocation, per-request authorization, derivation of role from the session) are not evidenced.	Owned by the separate `transportBackEndNative` system; to be covered by the backend submission. The client's sending of `roleId` / `statusId` makes this confirmation urgent. Risk accepted pending that evidence.
D2	No Content-Security-Policy and no security response headers (HSTS, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, COOP, CORP) exist in the repository.	Requires a hosting/server layer that is not part of this asset. Accepted as a documented deviation, not an unknown gap. Consequence: #16, #83, #85, #96, #98 remain open. Owner: platform/hosting team.
D3	No CI, dependency gate, or security lint baseline.	Remediation defined (§17). Risk accepted in the interim; the `axios` high-severity advisory is not yet fixed.
D4	Third-party PII egress via Photon place search and ui-avatars.	Functional requirement for address autocomplete and avatars. Mitigation is a backend proxy or local-only data; full fix in §17.
D5	Organizational items (2.1 G–H; 2.2 B; 2.3 B–D; all of 2.4; 2.5 A, F, K, O and others) have no evidence.	Outside engineering control; requires business action. Recorded as open, not claimed.
D6	Client profile (name, phone, email, role, none of it secret) held in cleartext `localStorage` while signed in.	Deliberate routing decision; the profile carries no token and is cleared on logout. The backend remains the authority.
D7	Single-factor authentication (phone + OTP); no second factor or device binding.	Accepted for the queue-console population on connectivity grounds. Server-side OTP rate limiting is a precondition (§17).
D8	`ProtectedRoute` fails open when the organization query errors, and the 365-day session lifetime for roles 11/12.	Fail-open is an availability decision for live queue operations; the long lifetime is backend policy. Both to be reviewed, with the risk recorded in §14 (#32, #76).

17. Security recommendations and roadmap

The remediation is phased by how much risk each phase removes for the effort it costs. Phases 1–4 are code-only and independent of the platform decision; Phase 0 and Phase 6 need hosting or backend action and carry the highest impact. The recommended order is Phase 1 → Phase 2 → Phase 3 → Phase 4, with Phase 0 started immediately in parallel.

Phase 0 — Server/header layer (highest impact; needs hosting access)

Put a CSP and security-header layer on `queue.dynamicsroute.tech`. The policy has to account for the inline JSON-LD block in `index.html` (hash or nonce, or move the block to an external file). The recommended header set: `Content-Security-Policy` (with `frame-ancestors 'none'`), `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, COOP/CORP, a `Cache-Control` policy (`no-store` on HTML, `immutable` on hashed assets), method restriction, and 404 blocks for `/.env`, `/.git` and `*.map`.

Phase	Scope	Blocks
0.1 Headers	CSP, HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, COOP, CORP	#16, #83, #85, #96, #98
0.2 Files & methods	404 blocks for `/.env`, `/.git`, `*.map`; method restriction; server identity suppression	#1, #2, #3, #92, #93
0.3 Caching	`no-store` on HTML, `immutable` on hashed assets	#12

Phase 1 — Stop the bleeding (half day, code-only)

#	Change	Annex B	Status
1.1	Gate `parseError` raw logging behind `import.meta.env.DEV`; log a terse, payload-free object in production	#4, #77	OPEN
1.2	Send the OTP as a string (`OTP: OTP`, not `Number(OTP)`)	#26	OPEN
1.3	Stop sending `roleId`/`statusId` on login and OTP, or reconcile the comments/README with reality	#28, #36	OPEN
1.4	Return only allow-listed server messages (no raw `message`/`error` passthrough) to close enumeration	#35	OPEN
1.5	`npm audit fix` and remove the unused `@tanstack/react-query`	#90	OPEN
1.6	Remove the dead `VITE_JWT_EXPIRATION_DAYS` variable	#76	OPEN

Phase 2 — Client hardening (1 day, code-only)

#	Change	Annex B	Status
2.1	Pin `build.sourcemap: false` explicitly for production	#6	OPEN
2.2	Change `ProtectedRoute` fail-open to distinguish transport failure (retryable message) from authorization failure (redirect)	#32	OPEN

#	Change	Annex B	Status
2.3	Add security linting (`eslint-plugin-security` or equivalent) with `detect-non-literal-regexp` enforced	#55	OPEN
2.4	Add tests for auth flow, route guards, 401 handling and socket unauthorized handling	#102	OPEN

Phase 3 — Third-party privacy (half day)

Host the Manrope font and the avatar generation yourself, and proxy Photon place search through the backend (or restrict it to the local Ethiopian-city list). That takes ui-avatars and Photon egress out of the picture and keeps third-party data flows to a minimum.

Phase 4 — CI, so this cannot rot again (half day)

Add a CI workflow: `npm ci` → `npm run lint` → `npm run build` → `npm test` → `npm audit --audit-level=high`. Add Dependabot for `npm` and secret scanning, and gate on a clean lint baseline. While you are in there, align the README's description of `npm run lint` with the actual ESLint script.

Phase 5 — Server policy source of truth (hosting)

Once a header layer exists, keep a single source of truth for the CSP (a `src/constants/csp.ts` consumed by both build and server configuration would work), plus a drift guard so the two copies cannot diverge. Add a post-build assertion that the meta tag, or the header, is actually present.

Phase 6 — Backend dependencies (separate repository)

Item	Annex B	Why
Confirm CSRF defence (double-submit or Origin checking)	#61, #62	Cookie-session authenticated mutations
Derive `roleId` / `statusId` from the session, never the body	#28	The client sends them for legacy reasons
Scope every queue/organization query to the cookie identity	#28	Ignore any client-supplied identifiers
Cookie flags: `HttpOnly; Secure`; correct `SameSite` for the deployment topology	#67–#70	Same-origin vs cross-site ambiguity must be resolved
Stop returning the JWT in the response body	#74	Target the cookie-only model
Server-side rate limiting on OTP and lockout	#37	Only a 60 s client cooldown exists
Server-side session regeneration, periodic expiry and revocation	#64–#66, #73, #76	365-day sessions and no regeneration are high-risk

Bottom line

Subject to the outstanding server, backend and organizational items above, the Dynamics Queue Admin console is suitable for continued operation and for INSA verification testing. The application-layer improvements marked PASS are real and can be checked independently in the source. This document does not claim the production header layer is live, because no server configuration exists in the repository to evidence it.

Document control, approvals and signatures

Document control

Item	Value
Document title	Dynamics Queue Admin: Security Documentation
Document identifier	`SEC-1.0.0`
Version	1.0.0 (initial submission)
Asset	Dynamics Queue Admin web console (asset code `ASSET-DYN-QA-001`)
Governing standard	INSA Secure Website Management Standard, 2014 EC
Status	For INSA review
Classification	Confidential, submitted to INSA

Revision history

Version	Date	Author	Summary
0.1	15 October 2026	Marew Masresha, General Manager	Initial draft security documentation.
1.0.0	15 October 2026	Marew Masresha, General Manager	First issue for INSA submission.

Approval

Role	Name	Signature	Date
Prepared by	Marew Masresha, General Manager	_____	15 October 2026
Reviewed by	Marew Masresha, General Manager	_____	15 October 2026
Approved by	Marew Masresha, General Manager	_____	15 October 2026

Prepared by — Marew Masresha, General Manager
Signature — _____ Date — 15 October 2026

Reviewed by — Marew Masresha, General Manager
Signature — _____ Date — 15 October 2026

Approved by — Marew Masresha, General Manager
Signature — _____ Date — 15 October 2026